

บทที่ 8

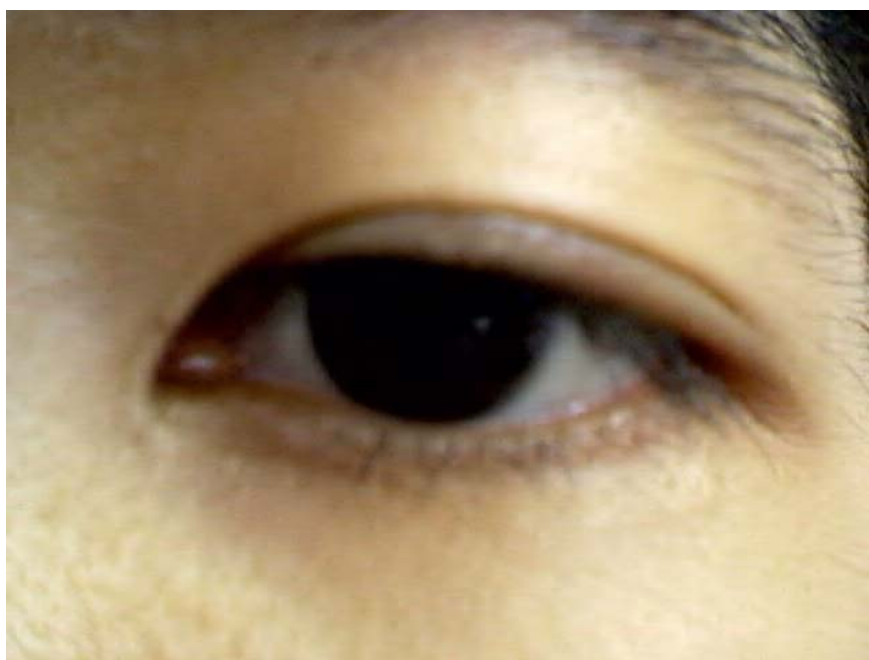
การหาจุดศูนย์กลางของดวงตา

ขั้นตอนการหาจุดศูนย์กลางเป็นสิ่งสำคัญที่สุดในการทำโปรแกรมควบคุมเมาส์ด้วยดวงตา เนื่องจากหากเราไม่สามารถหาตำแหน่งของดวงตาที่ถูกต้องแม่นยำแล้วเราก็จะไม่สามารถหาตำแหน่งที่ถูกต้องเพื่อที่จะควบคุมเมาส์ไปยังตำแหน่งที่ต้องการได้

โดยขั้นตอนการหาจุดศูนย์กลางของดวงตานี้เราจะเริ่มทดลองจากภาพนิ่งนามสกุล Bitmap ก่อนที่จะนำไปใช้กับภาพเคลื่อนไหวจริง โดยเราสามารถแบ่งขั้นตอนได้ดังนี้

8.1 ขั้นตอนการหาขอบบนขอบล่างของดวงตา

การที่จะหาจุดศูนย์กลางของดวงตานั้น ขั้นแรกจะต้องหาขอบบนขอบล่างของดวงตาก่อน สาเหตุที่ต้องหาขอบบนขอบล่างของดวงตาก่อนนั้นก็เพราะ หากเราหาบริเวณด้านซ้ายและขวาก่อนนั้นจะมีสิ่งรบกวนมากกว่า ตัวอย่างเช่น ผม เกาของหนังตา นั้นเป็นสิ่งสำคัญอย่างมากที่จะให้การหาจุดศูนย์กลางของตานั้นง่ายขึ้น (ขั้นต้นเรายังไม่มีการกำจัดสิ่งรบกวน เช่น ผม โดยการกำจัดสิ่งรบกวนนั้นจะกล่าวในบทถัดไป)

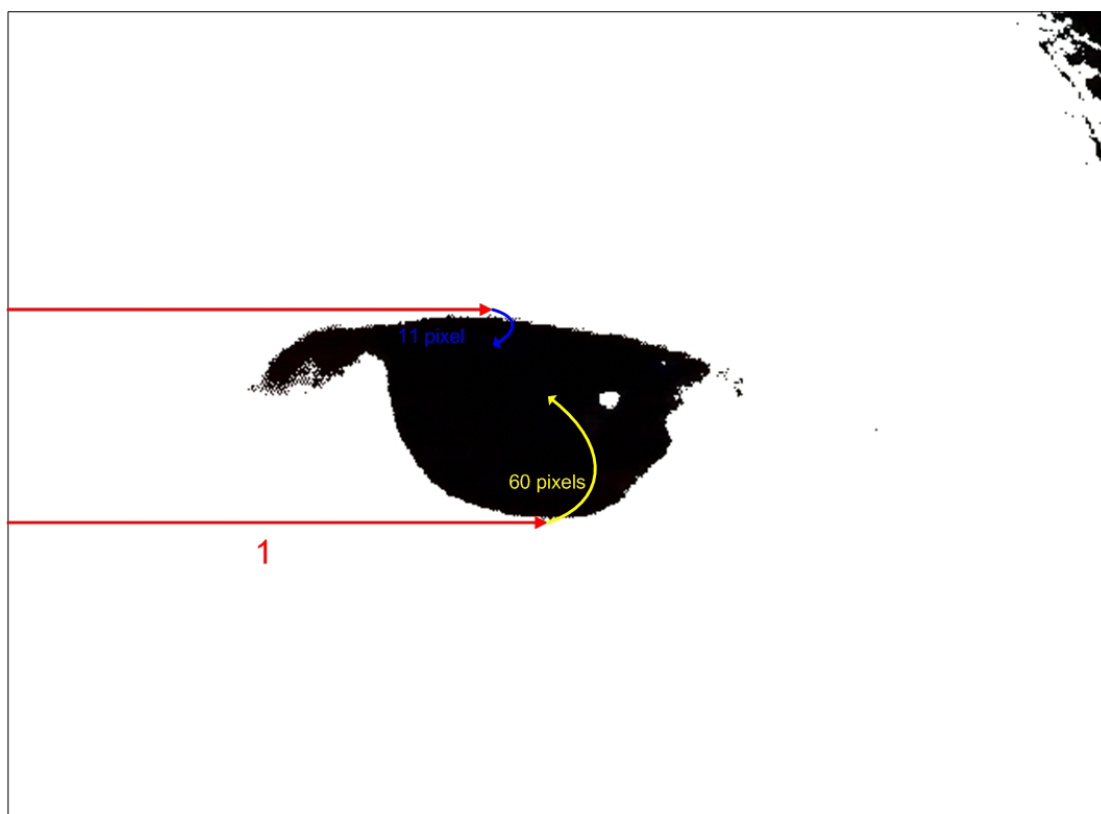


รูปที่ 8-1 แสดงภาพดวงตาที่สมบูรณ์ไม่มีเงาของแสงสว่างมารบกวน

ขั้นตอนตอนการหาขอบบนขอบล่างของดวงตานี้ เรานำภาพมาพิจารณาเฉพาะช่วงที่เราสนใจ (Threshold) โดยช่วงที่เราสนใจนั้นขึ้นพื้นฐานคือบริเวณที่เป็นสีดำ โดยมีค่า RGB คือ

1. ค่า R อยู่ในช่วง 0-30
2. ค่า G อยู่ในช่วง 0-30
3. ค่า B อยู่ในช่วง 0-30

โดยวิธีการนั้นเรียกว่า Similarity Segmentation หรือการทำให้กเมนต์เตชั่น จะตัดภาพเหลือแต่ช่วงสีที่เราสนใจนั้นคงจะเห็นได้จากรูปที่ 8-2



รูปที่ 8-2 ภาพดวงตาที่ผ่านการทำให้กเมนต์เตชั่น และการค้นหาขอบบนขอบล่างของดวงตา

ขั้นตอนการหาขอบบนขอบล่างจะแบ่งออกเป็น 2 ขั้นตอนย่อยคือ

8.1.1 หาขอบล่างของดวงตา

การที่เราจะเริ่มจากของล่างของดวงตานี้ก่อนเนื่องจากหากเราหาขอบบนของดวงตาก่อนเราจะเจอคิ้วซึ่งเป็นสิ่งรบกวนที่เราไม่ต้องการ

โดยเราจะทำการค้นหาตามแนวแกน X จากด้านล่างของภาพ จากล่างขึ้นบนเพื่อหาจุดแรกที่เป็นสีดำ โดยเราจะถือจุดแรกที่พบจะเป็นขอบล่างของดวงตา (สมมุติว่าด้านล่างของตาไม่มีสิ่งรบกวน)

8.1.2 หาขอบบนของดวงตา

หลังจากเราสามารถค้นหาขอบล่างของดวงตาได้แล้วนั้น เราจะทำการหาขอบบนของดวงตาโดยการค้นหาต่อจากขอบล่างของตาที่พบไปตามแนวแกน X จากล่างขึ้นบน โดยค้นหาแนวแกน X ที่ปราศจากสีดำทั้งแนวเราจะถือว่าแนวนั้นจะเป็นขอบบนของตาโดยเราอาจจะลดตำแหน่งของจากจุดที่สันนิษฐานว่าเป็นขอบบนลงมาได้อีก เนื่องจากสีดำที่เราพบเป็นจุดสุดท้ายนั้นอาจเกิดจากเงาของหนังตาก็ได้ ตามเส้นที่โยงจากขอบบนลงมาในรูปที่ 8-2

โดยขั้นตอนการหาขอบบนนั้นเราสามารถลดขั้นตอนการค้นหาได้คือ เราจะลดแนวการค้นหาจากล่างขึ้นบนโดยจะสันนิษฐานว่าตานั้นจะต้องไม่มีขนาดเล็กกว่าที่สันนิษฐานไว้ ซึ่งจะทำให้เราลดการค้นหาได้ “โดยจะลดการค้นหาตามแนวแกน Y โดยข้ามไป 60 Pixel (โดยจำนวนจุดที่ข้ามไปนั้นต้องสัมพันธ์กับความสูงของภาพที่นำมาพิจารณาด้วย)” ตามเส้นที่โยงขึ้นไปจากขอบล่างในรูปที่ 8-2 โดยจะไม่มีการค้นหาส่วนที่ข้ามนั้นเพื่อลดความเร็ว

8.2 ขั้นตอนการหาด้านซ้ายและด้านขวาของดวงตา

ในการหาด้านซ้ายและด้านขวาของดวงตานั้นเราจะใช้ Sliding Window โดยขนาดของ Sliding Window นั้นมีขนาดเป็นรูปทรงสี่เหลี่ยมจัตุรัส โดยที่เราสันนิษฐานว่าตาของเรานั้นมีขนาดสมมาตรเท่ากันทั้งความกว้างและความสูง

โดยเมื่อเราสามารถหาขอบบนและขอบล่างของดวงตาได้แล้วนั้นทำให้เรารู้ถึงขนาดของดวงตาที่สันนิษฐานไว้และช่วงขอบบนและขอบล่างที่ดวงตาอยู่ โดยเราจะใช้ Sliding Window ในการค้นหาตาที่อยู่ในช่วงนั้น โดยการค้นหานี้เราจะปรับปรุงวิธีการค้นหาเพื่อลดจำนวนการค้นหาให้น้อยลงที่สุดเท่าที่จะทำได้โดยมีรูปแบบการค้นหาตามการทดลองต่อไปนี้

วิธีแรกเริ่ม

เราจะเริ่มใช้ Sliding Window ตั้งแต่จุดแรกของแนวที่เราได้จากการหาขอบบนขอบล่าง โดย Sliding Window เริ่มค้นหาไปตามแนวแกนขอบบนขอบล่าง จากซ้ายไปขวา โดยภายใน Sliding Window เราจะค้นหาจุดทุกจุดที่เป็นสีดำที่อยู่ใน Window เอาไว้ โดยรวมค่าที่เอาไว้เพื่อนำมาเปรียบแต่ละ Window ที่ค้นหาตามแนวแกน X ว่า Window ไหนหากมีสีดำมากที่สุดจะถือว่า Window นั้นเป็นตา โดยเราจะได้ค่าด้านซ้ายและด้านขวาของตามาคำนวณกับขอบบนขอบล่างของตาเพื่อหาจุดศูนย์กลางของดวงตา

จะเห็นได้ว่าเราต้องทำการค้นหาทุกจุดตามแนวขอบบนขอบล่าง ซึ่งจะใช้ทรัพยากรในการประมวลผลของคอมพิวเตอร์จำนวนมากทำให้เราจึงต้องทำการปรับปรุงวิธีการค้นหา

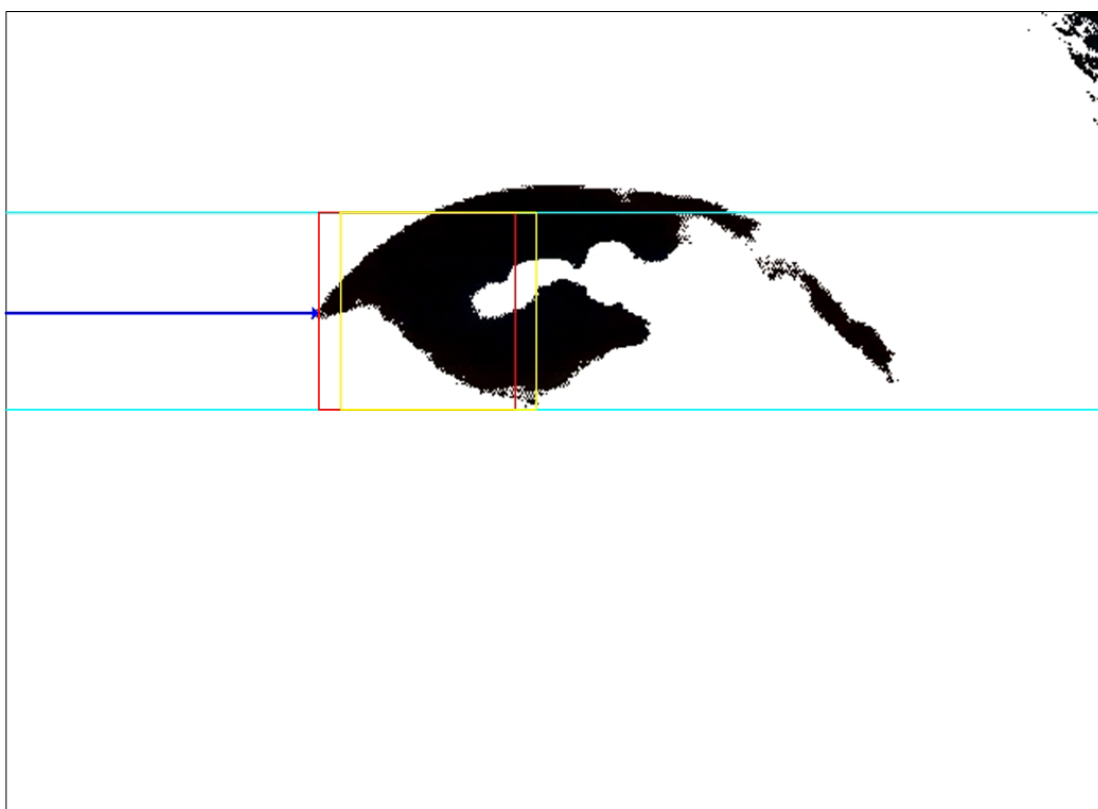
จากการทดลองหาใช้วิธีนี้ 1 ภาพ ใช้เวลาประมาณ 30-60 วินาที ซึ่งช้ามาก



รูปที่ 8-3 ภาพดวงตาที่มีเงาของแสงสว่างมารบกวนด้านขวาของตาดำ

ปรับปรุงจุดเริ่มต้นค้นหา

จากภาพที่ 8-4 แสดงให้เห็นว่าเราไม่จำเป็นต้องใช้ Sliding Window ตั้งแต่จุดเริ่มแรกของแนวขอบบนขอบล่าง ดังนั้นเราจะใช้เพียงการค้นหาเพียงแนวใดเพียงแนวหนึ่งบนแกน Y เพื่อหาว่าจุดใดควรจะเป็นจุดเริ่มที่จะใช้ Sliding Window



รูปที่ 8-4 ทำปรับปรุงจุดเริ่มต้นค้นหาและใช้ Sliding Windows เพื่อหาจำนวนจุดสีดำที่น่าจะเป็นตาดำ

จากการทดลองหลังจากปรับปรุงจุดเริ่มต้นของการค้นหา 1 ภาพ ใช้เวลาประมาณ 10-20 วินาที ซึ่งเร็วขึ้นมากทีเดียว

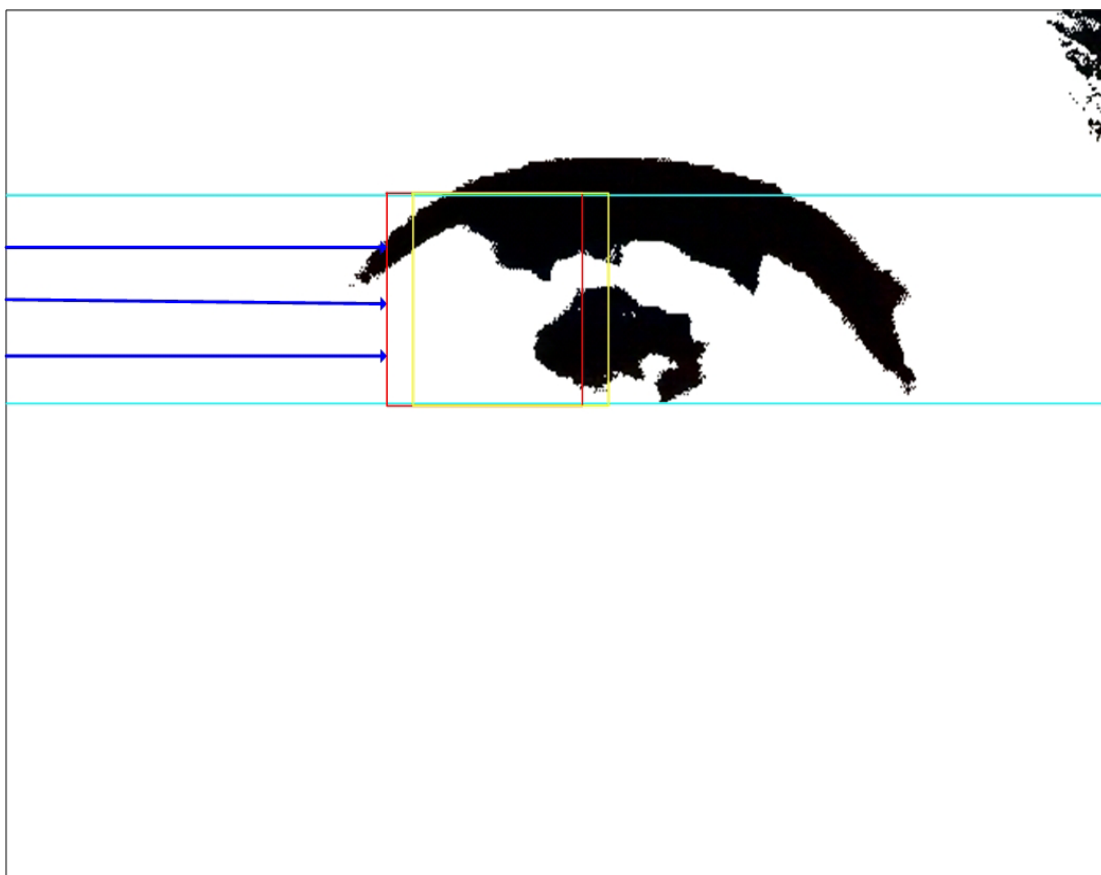


รูปที่ 8-5 ภาพดวงตาที่มีเงาของแสงสว่างมารบกวนทั้งด้านซ้ายและขวาของตา

หลังจากเราปรับปรุงเรื่องความเร็วเราก็ต้องคำนึงถึงความถูกต้องด้วยโดยตามรูปที่ 8-5 จะสังเกตเห็นได้ว่า ภาพนี้มีสิ่งรบกวนมากโดยมาจากแสงสะท้อนต่างๆ ซึ่งหลังจากการทำเซกเมนต์เตชันเราจะทำให้ได้ภาพดังรูปที่ 8-6

ซึ่งหากเราใช้เพียงแถว(Row) เดียวที่ใช้สำหรับการค้นหาจุดเริ่มต้นการใช้ Sliding Window อาจจะทำให้เกิดข้อผิดพลาดในการจุดศูนย์กลางของดวงตาได้ เราจึงทำการปรับปรุงการค้นหาโดยใช้ 3 แถว ในการค้นหาจุดเริ่มต้น ก่อนการใช้ Sliding Window ในการค้นหาตามรูปที่ 8-6

จากรูปที่ 8-6 แสดงให้เห็นว่าหากที่แสงสะท้อนบนตาเรามากทำให้ภาพที่เรารับนั้นมืดไป คือ ตาดำเรานั้นจะไม่ใช่สีดำ ซึ่งจะมีผลอย่างมาในการค้นหาจุดศูนย์กลางของตา



รูปที่ 8-6 ทำการปรับปรุงจุดก่อนเริ่มการใช้ *Sliding Window* เพื่อความถูกต้อง

นอกจากการปรับปรุงจุดเริ่มต้นค้นหาแล้วนั้นจริงๆแล้ว เราสามารถปรับปรุงส่วนของช่วงที่เราสนใจ(Threshold) ให้มีช่วงที่กว้างมากขึ้นก็ได้ แต่การที่เราเพิ่มช่วงมากขึ้นนั้นก็เป็นการเพิ่มสิ่งรบกวนเข้ามาเพิ่มตามไปด้วย ฉะนั้นจะต้องคำนึงถึงส่วนนี้ด้วย

ตามรูปที่ 8-7 แสดงให้เห็นว่าหากเพิ่มช่วงที่เราสนใจมากขึ้นก็จะมีสิ่งรบกวนเพิ่มมากขึ้นตามไปด้วยนั่นก็คือเงาของหน้าต่างนั่นเอง



รูปที่ 8-7 ทำการปรับค่า *Threshold* เพิ่มมากขึ้น แต่ก็ทำให้มีสิ่งรบกวนเพิ่มมากขึ้นตามไปด้วย

ปรับปรุงจุดสิ้นสุดการค้นหา

จากภาพที่ 8-6 แสดงให้เห็นว่าเราไม่จำเป็นต้องใช้ Sliding Window ค้นหาไปจนสุดของแนวขอบบนขอบล่าง หาก Sliding Window ของเรานั้นค้นหาไปจนจุดสีดำน้อยกว่าที่เราสมมุติฐานว่าจะไม่มีทางเป็นสีดำแล้ว ก็จะหยุดการค้นหา

จากการทดลองหลังจากปรับปรุงจุดสิ้นสุดของการค้นหา 1 ภาพ ใช้เวลาประมาณ 5-10 วินาที ซึ่งอาจจะลดลงมาไม่มาก แต่ก็ช่วยได้มากทีเดียว

ปรับปรุงโดยการหยุดการค้นหาโดยหากเจอ Window ใดมีจำนวนจุดสีดำเกินที่ตั้งไว้

เราอาจสามารถหยุดการค้นหาโดยหากมี Window ใดก็ตามที่มีจำนวนจุดสีดำเกินที่ตั้งไว้ให้สมมุติฐานได้เลยว่าเป็นสีดำ ซึ่งวิธีการนี้อาจไม่ใช่วิธีที่ดีเพราะอาจทำให้ความถูกต้องลดลงได้ แต่หากต้องการลดเวลาก็อาจเป็นทางเลือกหนึ่งที่น่าสนใจ เพราะว่าในแนวขอบบนขอบล่างอาจมีจุดที่มีสีดำสูงกว่าเกณฑ์ที่ตั้งไว้มากกว่าหนึ่งจุด ดังนั้นจุดที่มีสีดำมากที่สุดอาจไม่ใช่จุดนั้น

จากการทดลองหลังจากปรับปรุง 1 ภาพ ใช้เวลาประมาณ 3-8 วินาที ซึ่งลดลงมาไม่มาก เมื่อนำไปเทียบกับความถูกต้องที่ลดลง

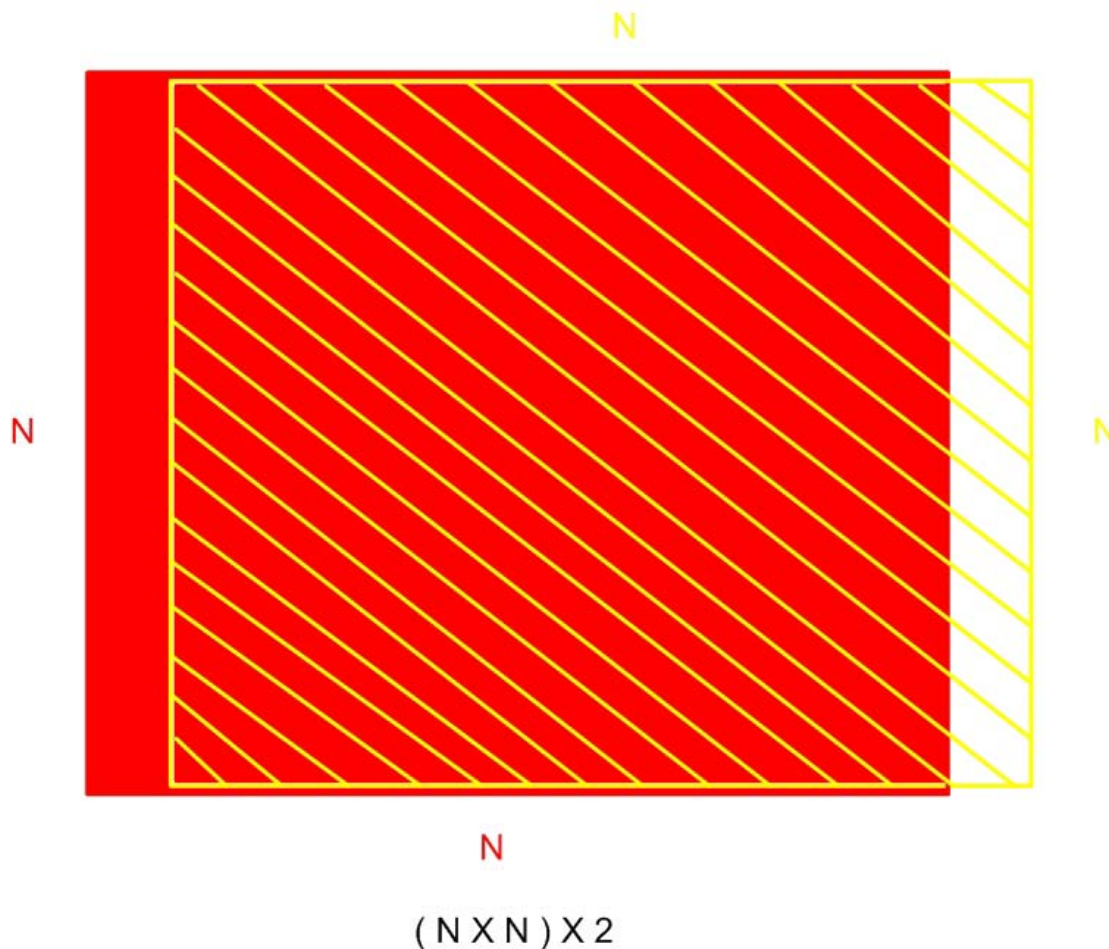
8.3 ปรับปรุง Algorithm ในการใช้ Sliding Window

จากการเขียนโปรแกรมในลักษณะเดิม เราได้มีการพิจารณาข้อผิดพลาดในการเขียนโปรแกรมว่า สาเหตุใด เป็นสาเหตุของความล่าช้าในการใช้ Sliding Window ซึ่งหากเราพิจารณาจากรูปที่ 8-8 จะพบว่า เรามีการค้นหาจุดที่เป็นสีดำซ้ำกันในแต่ละ Window โดยจะดูได้จากพื้นที่ส่วนที่ซ้อนทับกันของ Window หากเรามาลองคำนวณจุดที่ที่ใช้ในการค้นหาจะได้ว่า

Sliding Window มีขนาด $= N$

จำนวนจุดการค้นหาตามแนวขอบบนขอบล่างในภาพเดียว $= X$

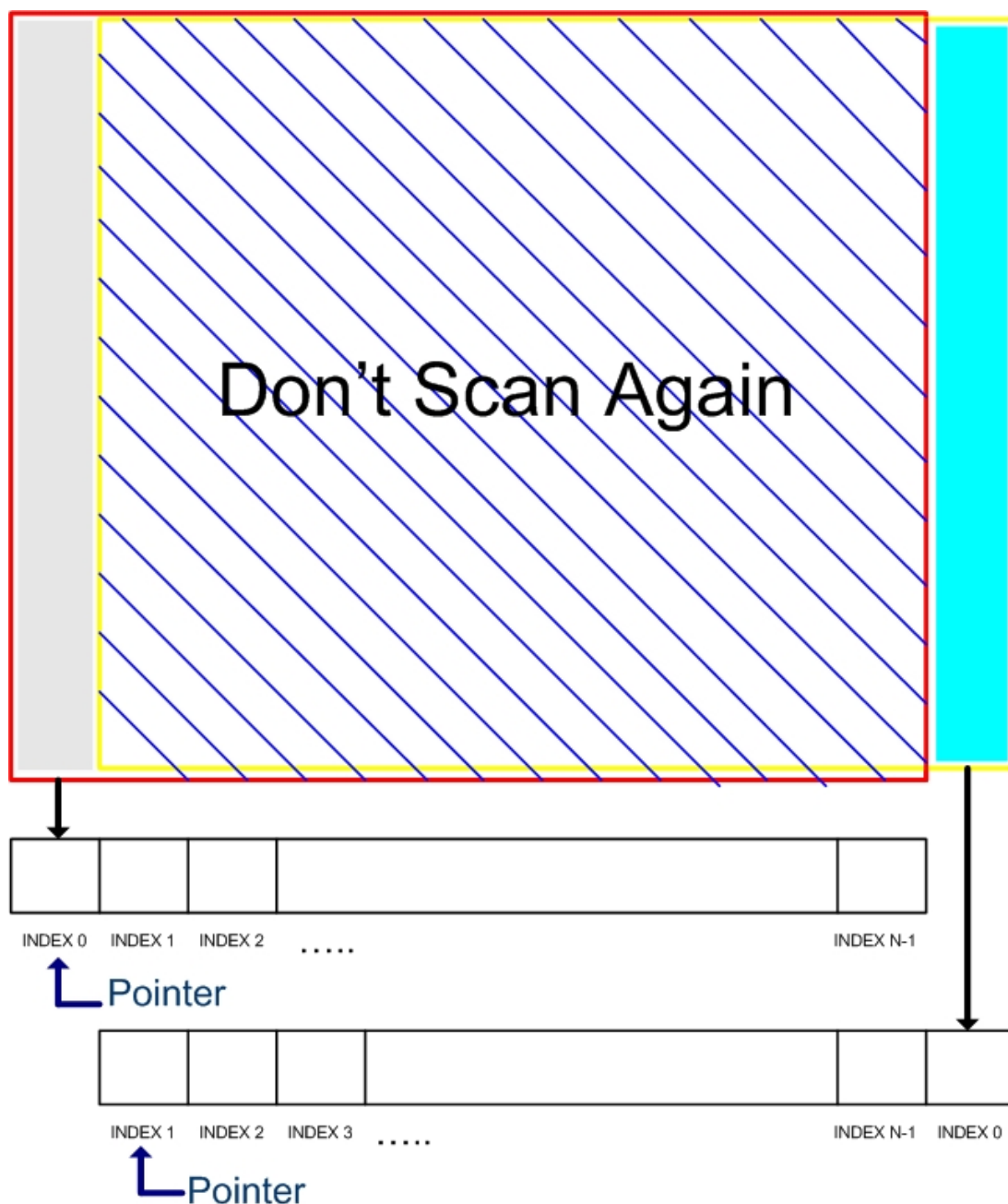
$$\text{จำนวนจุดที่ต้องค้นหา คือ } \sum_{i=1}^X (N^2)$$



รูปที่ 8-8 การค้นหาจุดสีดำของ Sliding Window แบบเดิม ซึ่งมีส่วนซ้อนทับกัน ทำให้เกิดความล่าช้า

เราทำการปรับปรุงโดยการสร้างบัฟเฟอร์ ตัวหนึ่งขึ้นซึ่งเป็น อาร์เรย์ 1 มิติ มีขนาดเท่ากับ ความกว้างของ Sliding Window เพื่อทำการเก็บผลรวมของจุดสีดำในแต่ละคอลัมน์ ซึ่งบัฟเฟอร์นี้เราจะมองใน ลักษณะของวงแหวน คือ แบบคิว (Queue) โดยจะมีตัวชี้ (Pointer) ชี้ไปยังคอลัมน์แรกของแต่ละ Window ดังเห็นได้จากรูปที่ 8-9

การค้นหาแบบนี้จะไม่ค้นหาจุดที่เคยค้นแล้ว โดยเราจะเก็บผลรวมของจุดสีดำในแต่ละคอลัมน์ไว้ในบัฟเฟอร์ หากต้องการหาผลรวมของจุดสีดำใน Window ก็ทำการค้นหาเพียง 1 คอลัมน์เท่านั้น แล้วนำใส่ลงไปบัฟเฟอร์ในคอลัมน์ที่ไม่อยู่ใน Window จากนั้นจะมารวมค่าที่อยู่บัฟเฟอร์ทั้งหมด ก็จะได้จุดสีดำทั้งหมดใน Window นั้น



รูปที่ 8-9 การค้นหาจุดสีดำของ *Sliding Window* แบบใหม่ ซึ่งจะไม่ต้องค้นหาส่วนที่เคยค้นหาแล้ว หากเรามาลองคำนวณจุดที่ใช้ในการค้นหาจะได้ว่า

Sliding Window มีขนาด $= N$

จำนวนจุดการค้นหาตามแนวขอบบนขอบล่างในภาพเดียว $= X$

จำนวนจุดที่ต้องค้นหา คือ $N \times X$

จะสังเกตได้ว่าจะลดจำนวนจุดที่ใช้ในการค้นหาลงเป็นจำนวนมาก ซึ่งช่วยลดเวลาในการค้นหาลงไปมาก

จากการทดลองหลังการปรับปรุงการใช้ Sliding Window 1 ภาพ ใช้เวลาประมาณไม่ถึง 1 วินาที ซึ่งเร็วขึ้นอย่างมาก

8.4 ขั้นตอนการหาจุดศูนย์กลางของดวงตา

หลังจากเราใช้ Sliding Window หาบริเวณที่เป็นของตาได้แล้วนั้น เราจะได้ค่าที่เป็นช่วงของ ด้านซ้ายและด้านขวาของตา เราจะนำมาคำนวณพร้อมกับขอบล่างขอบบนของตาเพื่อทำการหาจุดศูนย์กลางได้ดังนี้

$$\text{จุดตามแนวแกน X} = (\text{ด้านซ้าย} + \text{ด้านขวา}) / 2$$

$$\text{จุดตามแนวแกน Y} = (\text{ขอบล่าง} + \text{ขอบบน}) / 2$$

$$\text{จุดศูนย์กลางของดวงตา} = (\text{จุดตามแนวแกน X}, \text{จุดตามแนวแกน Y})$$